

Codex 任务电台

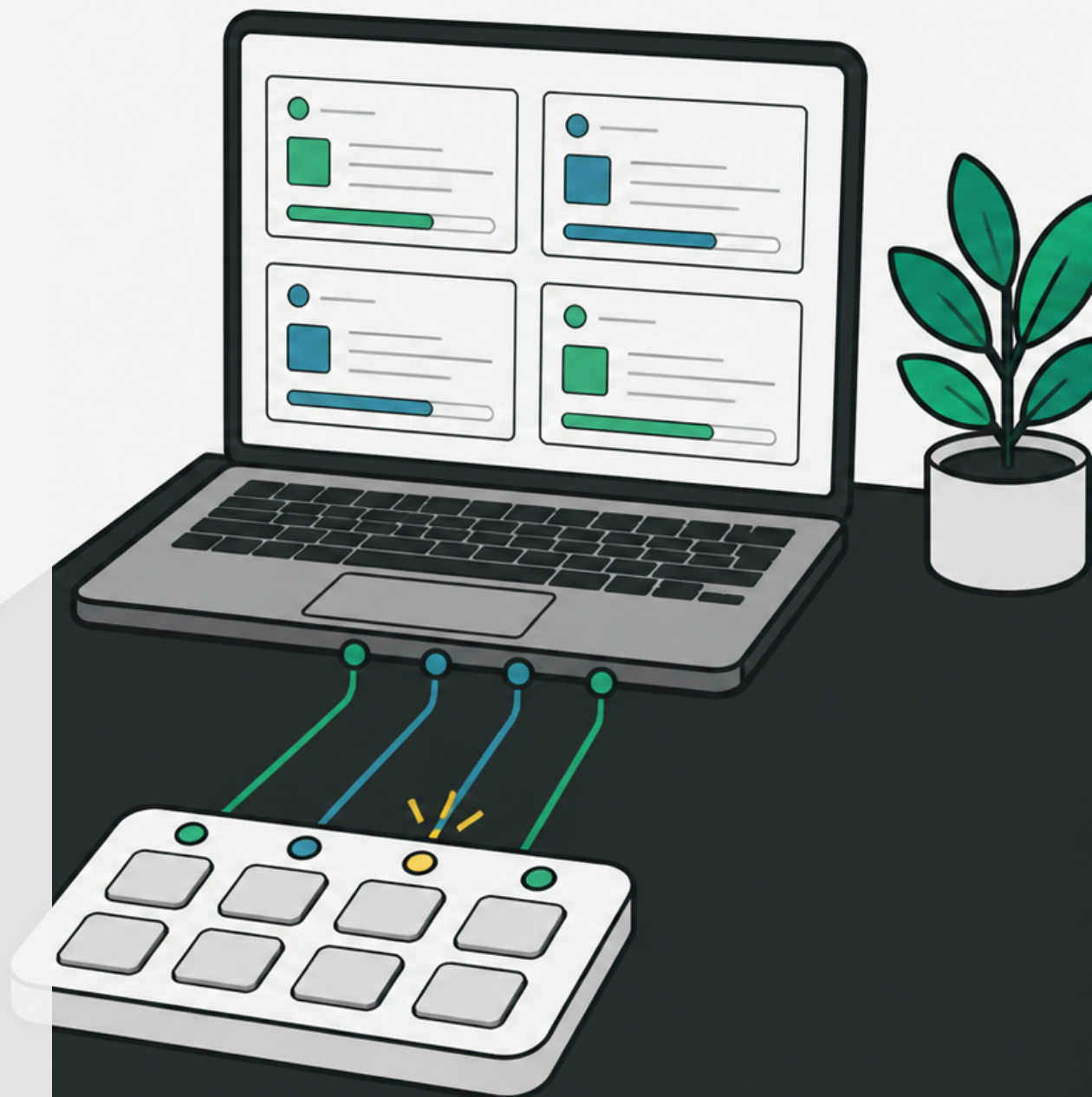
一把键盘，远程开发

Lark

知名外企嵌入式软件与 AI 产品转型负责人

背景为概念示意（AI 生成）

EasyInput V2 × Codex × 本地语音



Lark: 从嵌入式软件到 AI 产品

这次把固件、电脑端软件和模型服务接在一起，做一个自己会用的工具。



嵌入式软件

硬件、固件、系统边界



AI 工具实践

把模型放进真实工作流程



AI 产品

让人真的愿意每天用

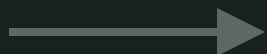
20 秒看懂它

真实演示只有四个动作。



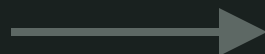
说

按住上排键



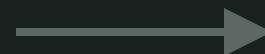
做

Codex 在后台



亮

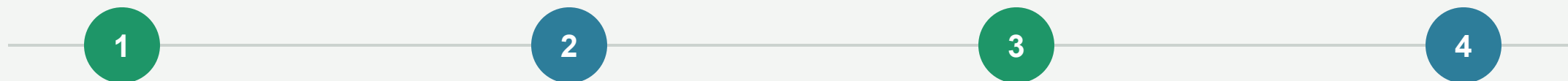
有新汇报



听

按一下下排键

把四个 Codex 任务，
变成四个可说、可听、会亮灯的实体频道。



常用要求直接说，需要检查代码时再回到电脑。

我每天同时开着好几个 **Codex** 任务

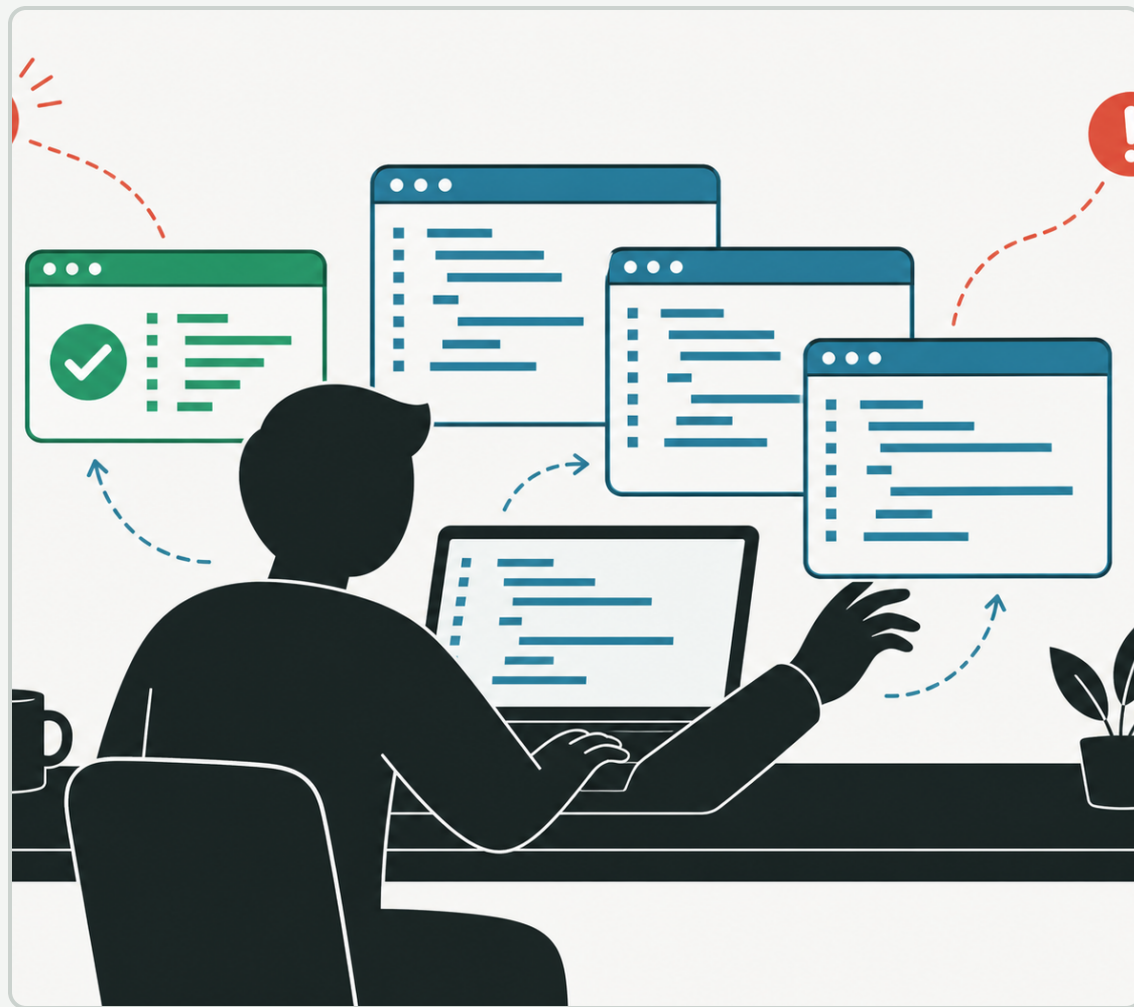
一个在改固件

一个在做 Mac App

一个在跑测试

还有一个在整理课程

**四个任务都能跑，
但我一直在来回看。**



工作场景示意（AI 生成），非现场照片

打字、切窗口、等结果，反复打断工作



总在打字

很多要求只要说一句，
却要先坐回电脑前。



总在切窗口

看谁做完了，
看谁还在等我。

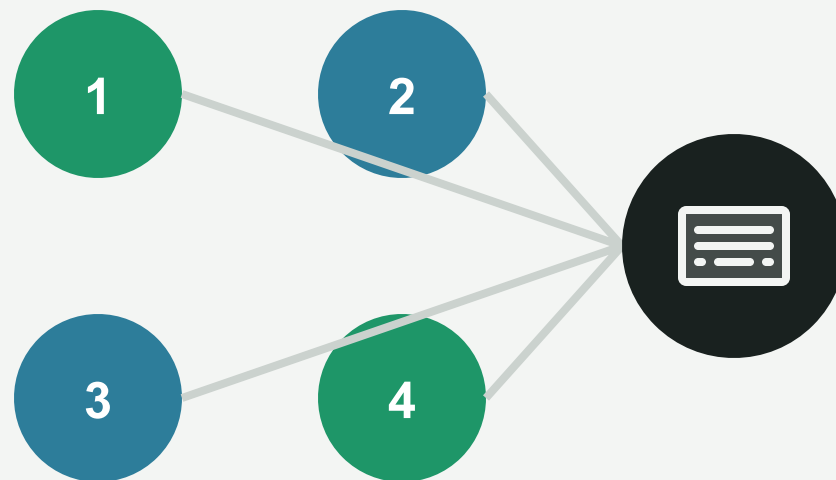


总在等结果

盯着进度，
反而没去做别的事。

AI 能并行， 人却只有一双手。

真正卡住的，是下要求、看状态、收结果。

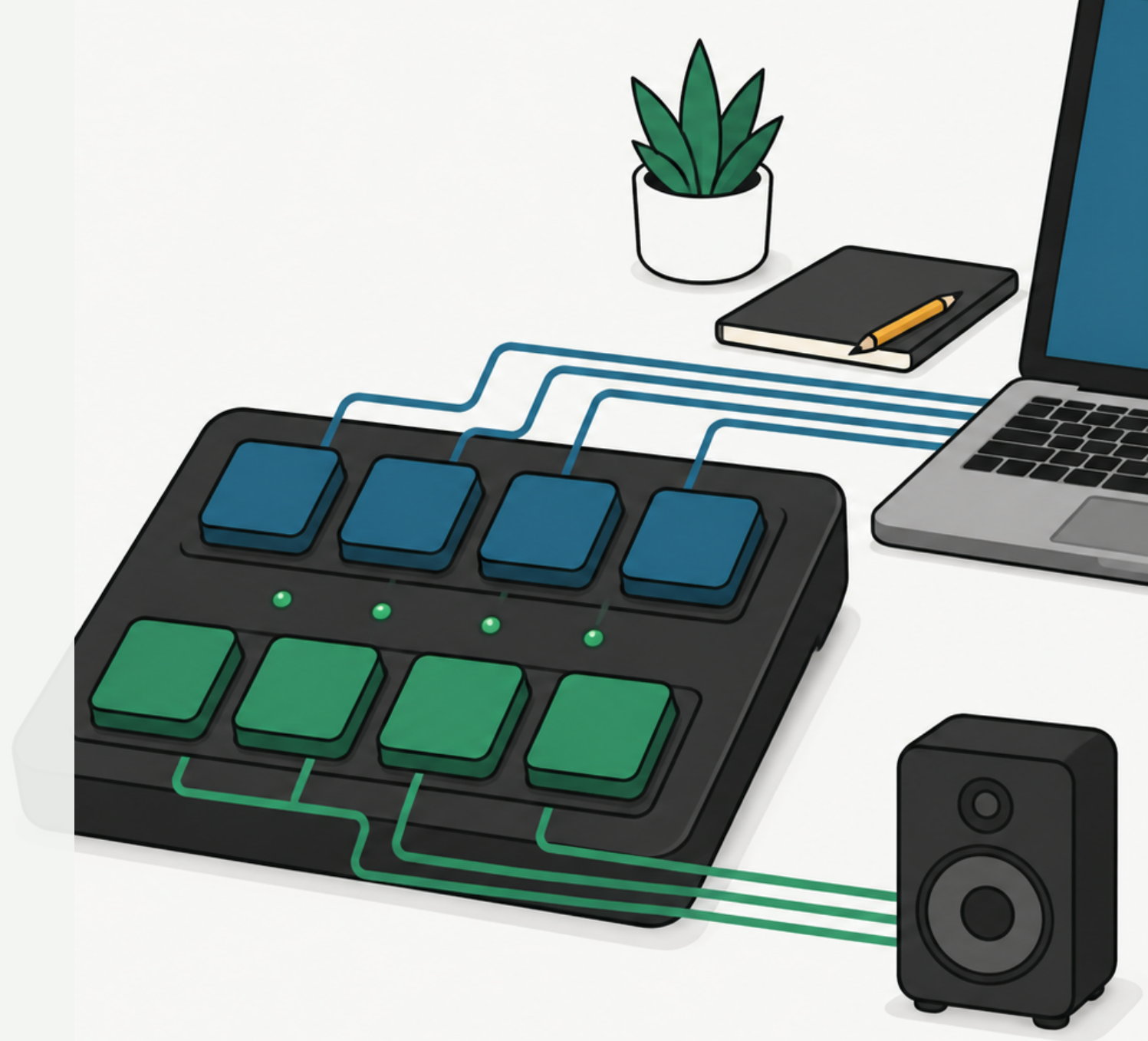


手边正好有一块 八键开发板

八个键、五颗灯、麦克风、
Wi-Fi，还有扬声器链路。

不如把它改成任务对讲机。

概念示意（AI 生成），实物见下一节



四个任务，就是四个频道

我不需要一直看它，只要知道该对哪个频道说、听哪个频道。



CH 1

固件任务



CH 2

App 任务



CH 3

测试任务



CH 4

课程任务

最初的交互草图，最后基本没变



S1-S4
按住说要求

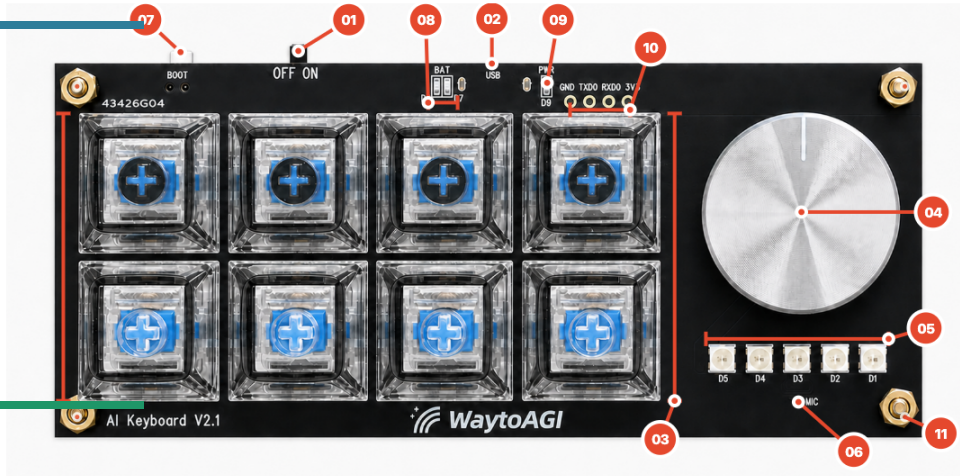


S5-S8
按一下听汇报

EASYINPUT · FRONT COMPONENT GUIDE

认识开发板正面

从日常操作，到状态、恢复与调试入口



第一次使用先认识 01-06。BOOT 与串口触点只在烧录、恢复或进阶调试时使用；BOOT 不是蓝牙配对键。

EASYINPUT V2.0 · ASSEMBLED FRONT VIEW

FRONT COMPONENTS / 2026-07-24

COMPONENT INDEX

正面部件索引

日常操作与连接

- 01 电源开关
拨到 ON 开机，OFF 关机
 - 02 USB-C 接口
供电、数据连接与固件烧录
 - 03 机械按键 S1-S8
八个可配置的实体输入
 - 04 旋钮 S9
支持旋转、短按与长按
 - 05 RGB 彩色灯 D1-D5
灯光反馈、动画与课程实验
 - 06 麦克风收音孔
使用键盘麦克风时请勿遮挡
- ##### 状态、恢复与调试
- 07 BOOT 下载 / 恢复键
固件烧录或设备恢复时使用
 - 08 充电状态灯 D6 / D7
显示充电相关状态
 - 09 独立状态灯 D9
硬件已接，当前固件尚未启用
 - 10 串口调试触点
GND / TXD0 / RXD0 / 3V3

安装结构

- 11 四角固定柱
用于安装底座或外壳



左四灯 = 信箱
最右灯 = 运行数



旋转：逆时针减 · 顺时针加（板载扬声器）
短按：播报当前音量

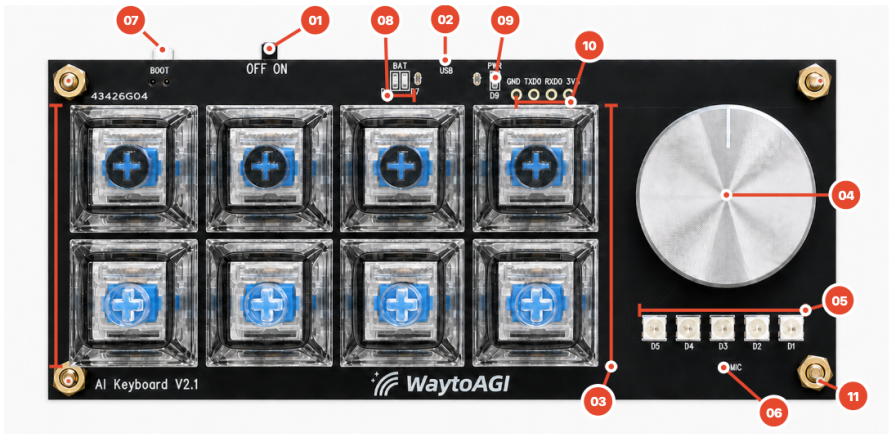
同一块开发板， 换一套用途

这次改软件，不重新焊硬件。

固件控制按键、录音和播放。
Mac 负责连接 Codex 任务。

EASYINPUT · FRONT COMPONENT GUIDE 认识开发板正面

从日常操作，到状态、恢复与调试入口



第一次使用前先认识 01-06。BOOT 与串口触点只在烧录、恢复或进阶调试时使用；BOOT 不是蓝牙配对键。

EASYINPUT V2.0 - ASSEMBLED FRONT VIEW

01-11

COMPONENT INDEX 正面部件索引

日常操作与连接

01 电源开关

拨到 ON 开机，OFF 关机

02 USB-C 接口

供电、数据连接与固件烧录

03 机械按键 S1-S8

八个可配置的实体输入

04 旋钮 S9

支持旋转、短接与长按

05 RGB 彩色灯 D1-D5

灯光反馈、动画与课程实验

06 麦克风收音孔

使用键盘麦克风时请勿遮挡

状态、恢复与调试

07 BOOT 下载 / 恢复键

固件烧录或设备恢复时使用

08 充电状态灯 D6 / D7

显示充电相关状态

09 独立状态灯 D9

硬件已接，当前固件尚未启用

10 串口调试触点

GND / TXD0 / RXD0 / 3V3

安装结构

11 四角固定柱

用于安装底座或外壳

FRONT COMPONENTS / 2026-07-24

EasyInput V2 实物部件图 · 保留完整视野

用人话认识这块硬件



能按

8 个机械键



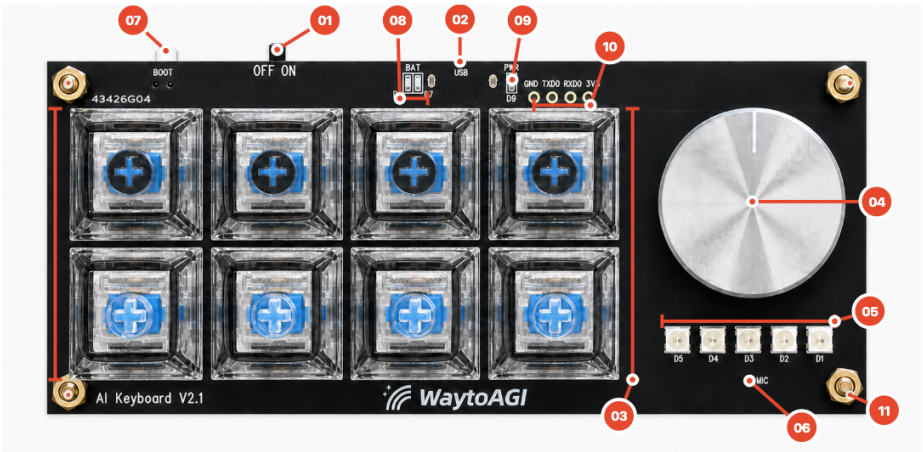
调板载音量

10 档 · 短按播报

EASYINPUT · FRONT COMPONENT GUIDE

认识开发板正面

从日常操作，到状态、恢复与调试入口



第一次使用先认识 01-06。BOOT 与串口触点只在烧录、恢复或进阶调试时使用；BOOT 不是蓝牙配对键。

EASYINPUT V2.0 - ASSEMBLED FRONT VIEW

01-11

COMPONENT INDEX

正面部件索引

日常操作与连接

- 01 电源开关
拨到 ON 开机，OFF 关机
- 02 USB-C 接口
供电、数据连接与固件烧录
- 03 机械按键 S1-S8
八个可配置的实体输入
- 04 旋钮 S9
支持旋转、短按与长按
- 05 RGB 彩色灯 D1-D5
灯光反馈、动画与课程实验
- 06 麦克风收音孔
使用键盘麦克风时请勿遮挡

状态、恢复与调试

- 07 BOOT 下载 / 恢复键
固件烧录或设备恢复时使用
- 08 充电状态灯 D6 / D7
显示充电相关状态
- 09 独立状态灯 D9
硬件已接，当前固件尚未启用
- 10 串口调试触点
GND / TXD0 / RXD0 / 3V3

安装结构

- 11 四角固定柱
用于安装底座或外壳

FRONT COMPONENTS / 2026-07-24



能听

I2S 麦克风



能说

功放与扬声器



能亮

5 颗 WS2812

再加上 USB、蓝牙和 Wi-Fi，它就能和电脑一起工作。

音频链路用到了这些硬件

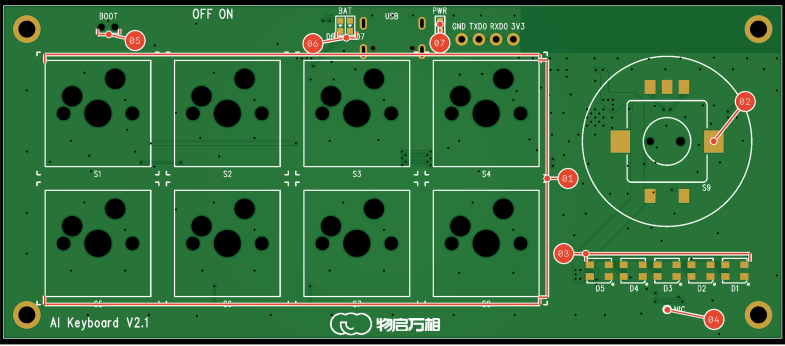
EASYPINUT - HARDWARE OVERVIEW

PCB 正面：按键、旋钮与状态反馈

先认识安装位置与可见标识，再把它们和手里的装配部或开发板对应起来

PCB FRONT / PHYSICAL LAYOUT

用户操作与反馈部件的安装位置



FIRST LOOK / 先记住

拿到开发板后，日常最常接触的是八个按键、旋钮、RGB 灯和麦克风收音孔。BOOT 主要用于固件下载与恢复，不是蓝牙配对键。

01 / 02

本页主要回答

正面哪些位置会安装你能够按下、旋转或观察的部件？

位置索引 / LOCATION INDEX

01 按键安装位 S1-S8

两排四列，共 8 个机械按键安装位置

02 旋钮编码器安装位 S9

支持旋转和按压，动作可由固件与 App 配置

03 RGB 彩色灯 D1-D5

用于状态反馈、灯光动画和课程实验

04 麦克风收音孔

声音从此处进入，麦克风器件安装在背面

05 BOOT 下载 / 恢复按键位

用于烧录、恢复和教程，不用于日常输入

06 充电状态灯 D6 / D7

位于 BAT 标识下方，显示充电相关状态

07 独立状态灯 D9

由固件控制的独立状态 / 诊断灯

读图提示 / READING NOTES

■ 本页是未装配 PCB 布局图，收到的开发板已经装好按键、旋钮与其他元器件。

■ 默认键位功能请查看单轴的“按键位置与功能速查图”。

DOCUMENT CONTROL

PRODUCT

Easypinut V2.0

PCB MARKING

AI Keyboard V2.1

DOCUMENT

PCB-OVERVIEW v1.0 · 2026-07-23

PSRAM

放音频缓冲

Wi-Fi 音频

和 Mac 传声音

EASYPINUT - HARDWARE OVERVIEW

PCB 背面：主控、存储与音频电路

ESP32-S3R8 · 8 MB 封装内 PSRAM · 16 MB 外置 Flash · USB-C · 电池与音频

PCB BACK / CORE HARDWARE

核心器件、连接接口与声音链路

第一版先把本地 Wi-Fi 跑通

键盘和 Mac 在同一个网络里，先把声音、状态和播放做扎实。



这里的“远程”是离开 **Codex** 窗口，不是已经完成公网遥控。

Mac App 负责绑定四个任务

真实界面截图，任务名称已换成课程示例。

先选任务，再确认绑定

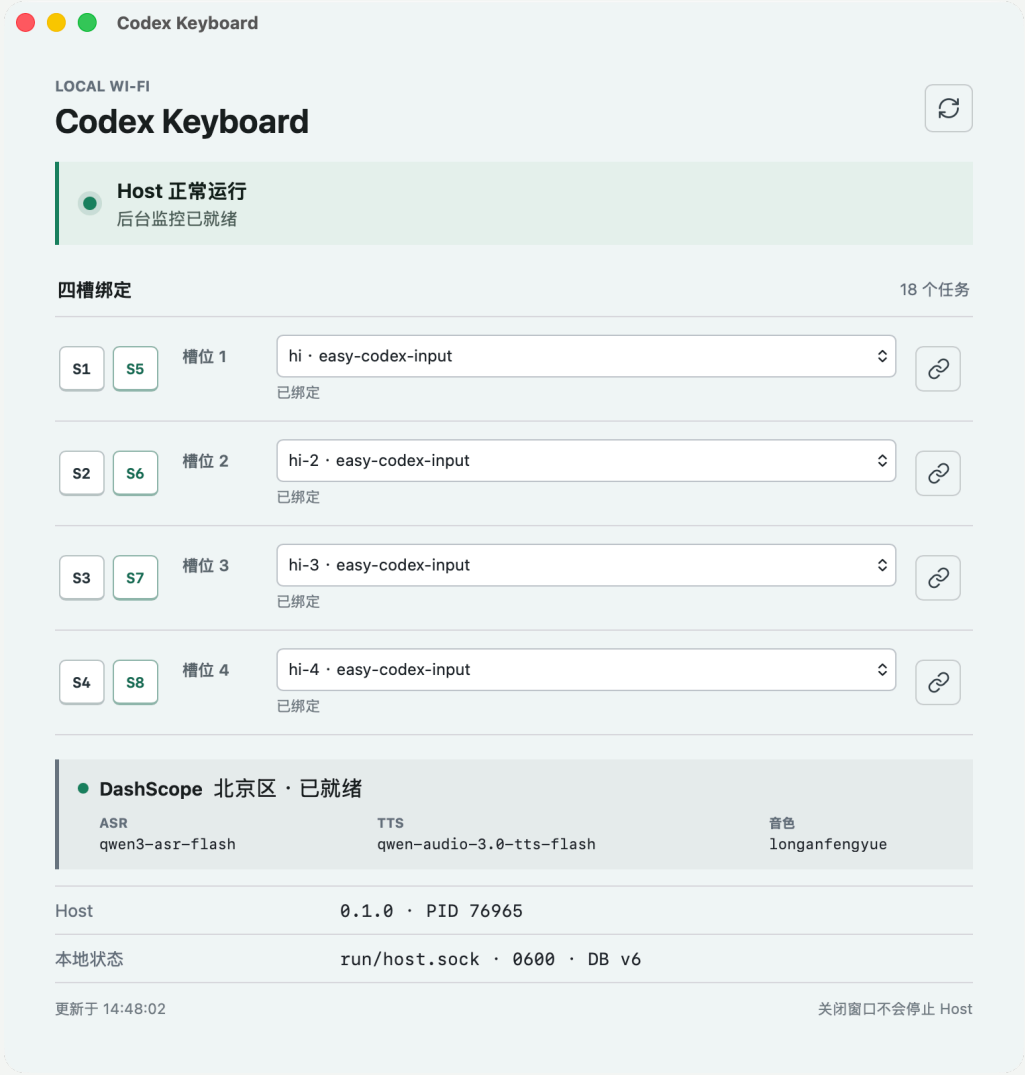
槽 1 对应 S1 / S5。

可查看排队数量和未听次数。

其余槽按同样方式绑定。

Host 正常不代表键盘已连通。

这个界面没有独立设备在线探针。

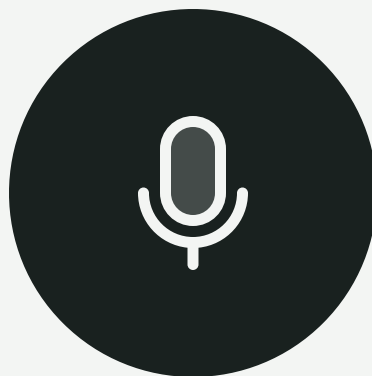
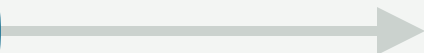


按住说，松开就发



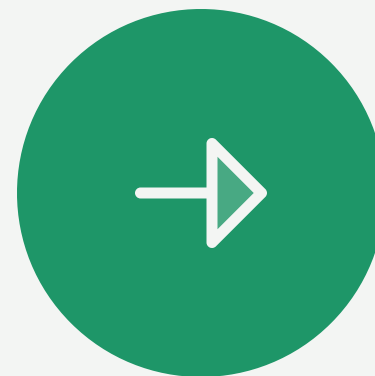
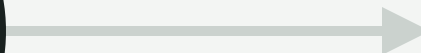
按住 S1-S4

选中对应任务



直接说要求

录音先经局域网到 Mac



松开后发送

ASR 结果进入 Codex

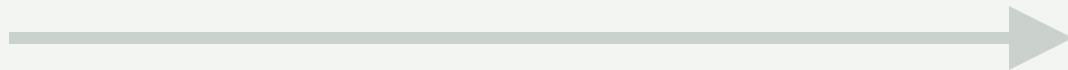
发完就走。

Codex 在后台干活。

这才是我真正想省下来的时间：不用守着屏幕。



一句要求



一段完整工作

前四颗看信箱，第五颗看忙不忙

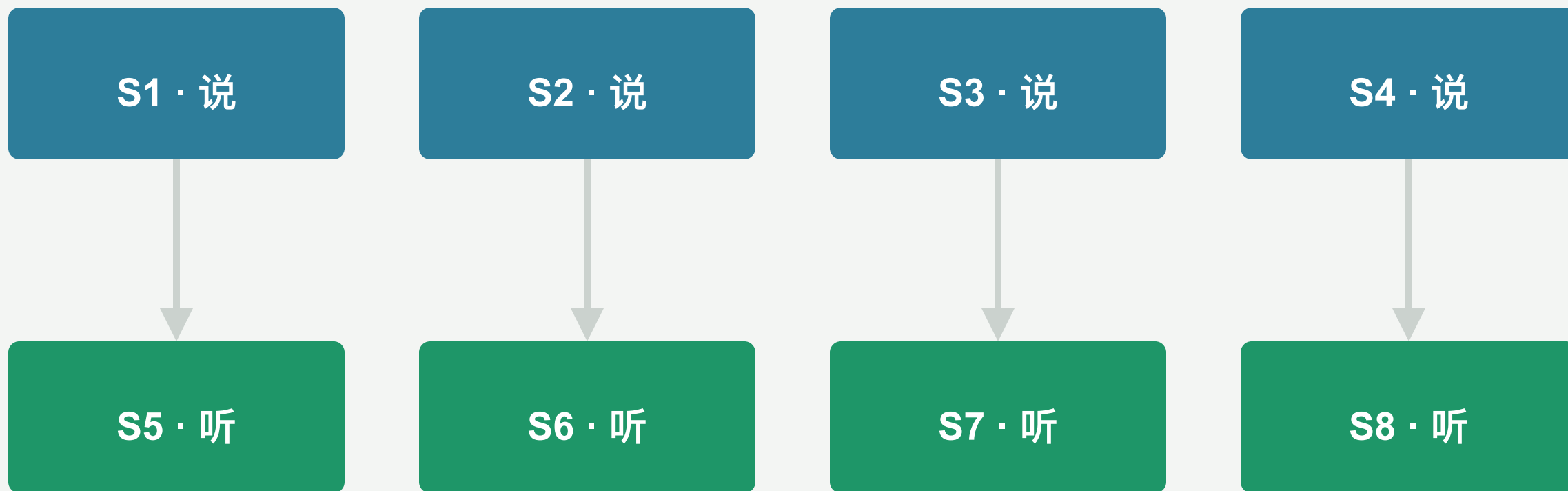
每槽独立调亮，和槽号无关；右图示例为 3 个不同任务正在运行。



信箱灯 同一槽累计未听的完成次数越多，这一颗越亮。



S5-S8：按一下，听对应任务汇报



一组键位就是一个频道：上排发消息，下排收回信。

三个角色，分工很简单



原理基础篇：十张概念卡

复现时每一步背后都有一个「为什么」。名词看不懂，随时回这一章查。



局域网直连

卡 1



配网

卡 2



Flash 与 NVS

卡 3



声音数字化

卡 4



认证与加密

卡 5



代际

卡 6



FIFO 排队

卡 7



任务完成

卡 8



听过语义

卡 9



常驻管家

卡 10

本地直连，仍然需要云端模型

分清设备通信和模型调用，才不会误以为整套系统离线可用。

局域网



互联网服务

录音 → DashScope ASR

任务与总结 → Codex

总结文本 → DashScope TTS

省掉的是公网中继，不是所有云端调用。

断外网后，新的识别、任务和合成会受影响。已有缓存需 Host 与局域网仍正常才能播放。

本项目沿用 UDP；WebSocket 也可用于局域网，不是单片机不能用。

先保存连接信息，再检查能否通信

本课的 provision-lan 命令走 USB HID，不会失败后自动改走蓝牙。



Mac 下发配置

SSID、密码、Mac IPv4、UDP 端口、设备密钥



键盘保存到 NVS

收到精确保存确认，命令才报告 provisioned



验证局域网链路

检查心跳、任务状态变化，再做一次语音输入

三件要留意的事

密码从标准输入读取

不写入命令参数、截图或日志

Mac 地址可能变化

更换网络后核对保存的端点

保存成功不等于连通

AP 隔离和防火墙仍可能阻断

刷哪一段，保留哪一段

以本版 partitions.csv 为准。NVS 在应用之前，不在应用后面。

起始地址	内容	本次三段烧录
0x0000	bootloader 引导	写入
0x8000	分区表	写入
0x9000	NVS 配置	保留
0xF000	PHY 数据	保留
0x10000	应用程序（3 MB）	写入
0x310000	sound_a 音效区	保留
0x3A0000	sound_b 音效区	保留



NVS 里保存什么

Wi-Fi、Mac 端点、设备密钥、音量档位。

音量播报在哪里

十条提示音嵌入应用，不在独立音效区。

不执行整片擦除。更换分区布局时，必须重新评估配置和音效的保留方式。

录音和播报，用两套音频参数

PCM16 是采样数据。采样率、位深和声道数要一起核对。

上行：你说话

16 kHz · 单声道 · PCM16



下行：听总结

48 kHz · 单声道



HMAC 校验上行来源和完整性，不隐藏 PCM 内容。声音变调时，先核对采样率。

认证检查身份，加密隐藏内容

两者解决不同问题。仅有 HMAC 的报文，不应被称为密文。



HMAC-SHA256

检查来源和数据是否被改动

用于上行 PCM、控制与状态报文。

认证通过后，仍需检查会话和顺序。

不负责隐藏上行录音内容。



AES-256-GCM

加密并校验音频内容

用于下行音频与本地密文缓存。

下行解密成功后，也要核对播放身份。

局域网直连不替代这些检查。

设备通信密钥与 Mac 缓存密钥分开保存。公开课不要展示密钥、录音或真实任务内容。

重新连接，不会自动删掉未听内容

摘要版本回答“哪段内容”，播放租约回答“哪一次播放”。

示例	摘要版本	播放身份	Host 如何处理
第一次播放	7	A	未播完，仍为未听
断电后重连	7	B	同一内容，重新请求
A 的迟到回执	7	A	不能算作 B 已听完
B 完整播放	7	B	身份匹配，标记已听

新摘要完整发布后才替换旧版。这里的 A/B 是教学示例，实际还会核对连接编号和租约。

一个任务排一队，最多四个任务同时跑

队列按任务划分；两个槽绑定同一个任务，也不会让这个任务并发执行。



● 示例：3 个任务运行，第五灯为紫色；排队数量不算运行数量。

任务结束后，还要等总结和语音生成

task_complete 是触发点。进程退出、暂时没输出，都不能代替完成事件。

1

观察到任务完成

更新运行数，读取本次助手最终回复。

2

Luna High 重新提炼

带上旧未听内容；长度跟随信息量。

3

TTS 和缓存发布完成

对应信箱灯亮起，现在才有音频可听。

完成 ≠ 做成功

最终回复也可能报告失败、限制或等待你处理的问题。

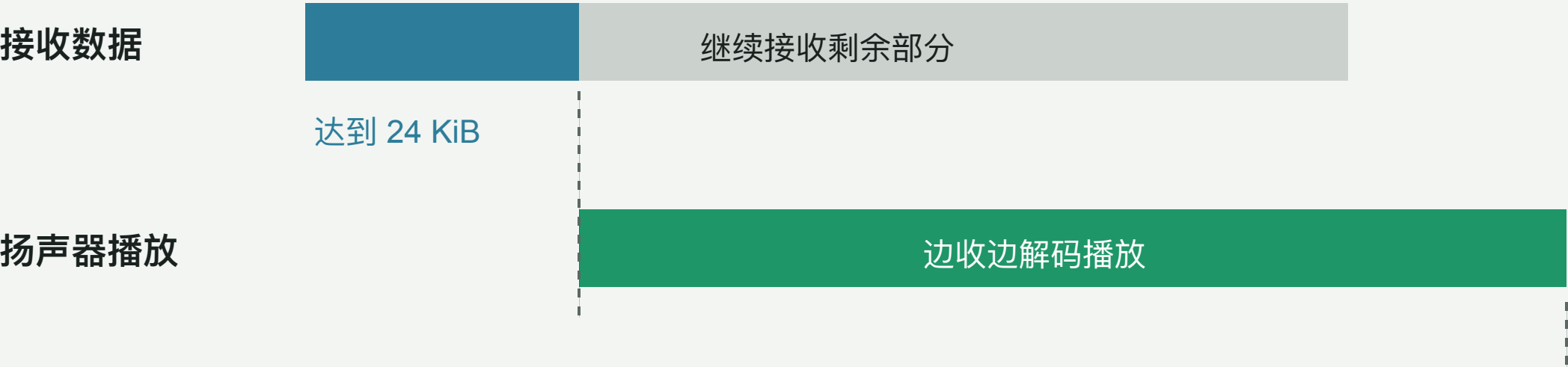
没有固定 15 秒

短确认不扩写，复杂结果多讲。
当前单条音频上限为 150 秒。

灯没有亮时，按完成事件、总结、TTS、缓存发布的顺序查，不先认定是按键坏了。

先播起来，播完确认后再清理缓存

整段分配 PSRAM，不等于整段收完才播。当前已经使用预缓冲播放。



短于预缓冲的音频，收完后播放。
慢网仍可能等待或失败，不能保证不断音。

播放完成 + 匹配身份的回执
Host 才标记本条已听

PTT 抢占或断电时保留未读。拨电后物理灯会灭，重连后再同步仍未听的信箱状态。

关掉 App 窗口，Host 继续工作

Host 是用户登录后运行的 LaunchAgent。桌面窗口不是后台服务本身。



桌面 App

选择并绑定任务

查看队列与未听次数



easy-codex-host

观察任务、调用模型

维护 SQLite 与音频缓存



键盘

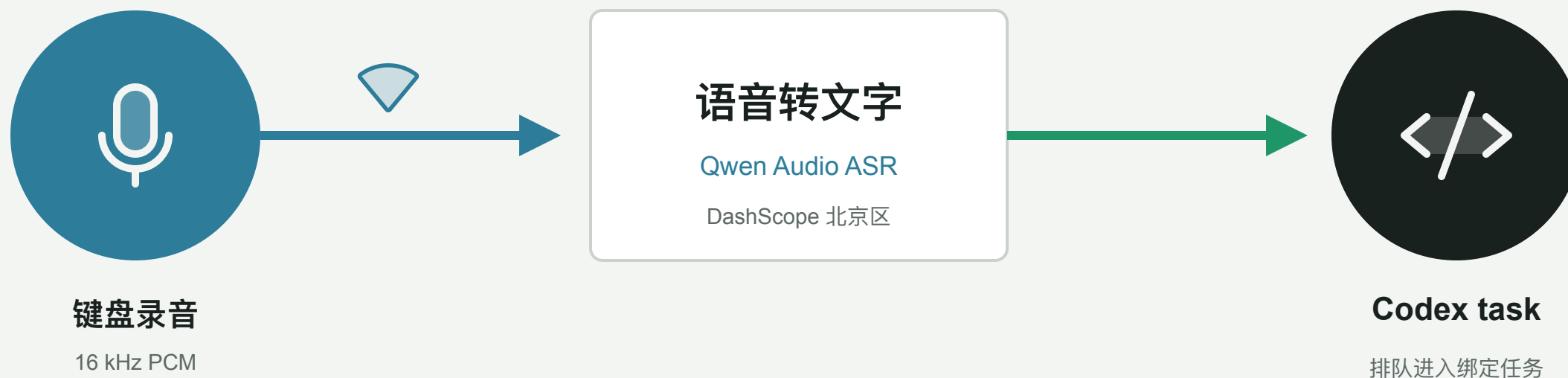
发送槽位动作

显示灯光、播放声音

Mac 睡眠、退出登录或断网，仍可能中断工作。

升级 Host 使用 launch-agent-upgrade；仅重启桌面 App 不会更换后台程序。

说进去：麦克风 → 千问 **ASR** → **Codex**



键盘不理解内容。它只负责把声音可靠地送到 Mac。

做完后：Codex → Luna High 总结

只总结最近几次助手的最终回复，不把过程日志念给人听。

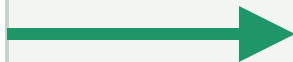


任务完成事件

拿到助手最后的答复

保留完成了什么

保留结果和待办



口播示例，不是真机验收记录

“已改好表单校验，空姓名现在不能提交。
单元测试通过了，手机上还需要你试一次。”

上次没听的内容，也交给 Luna 一起整理。

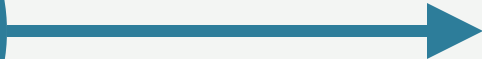
说清改动、验证结果，以及还需要人做什么。

听回来：千问 **TTS** → 本地缓存 → 扬声器



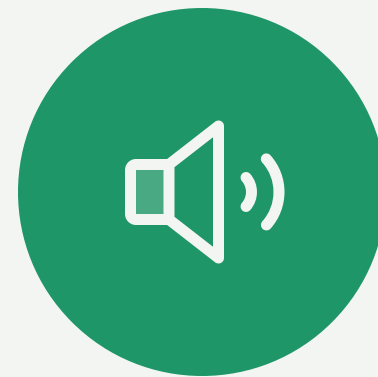
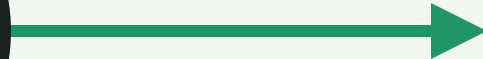
千问 TTS

qwen-audio 3.0 tts flash



Mac 本地缓存

先生成好，等按键

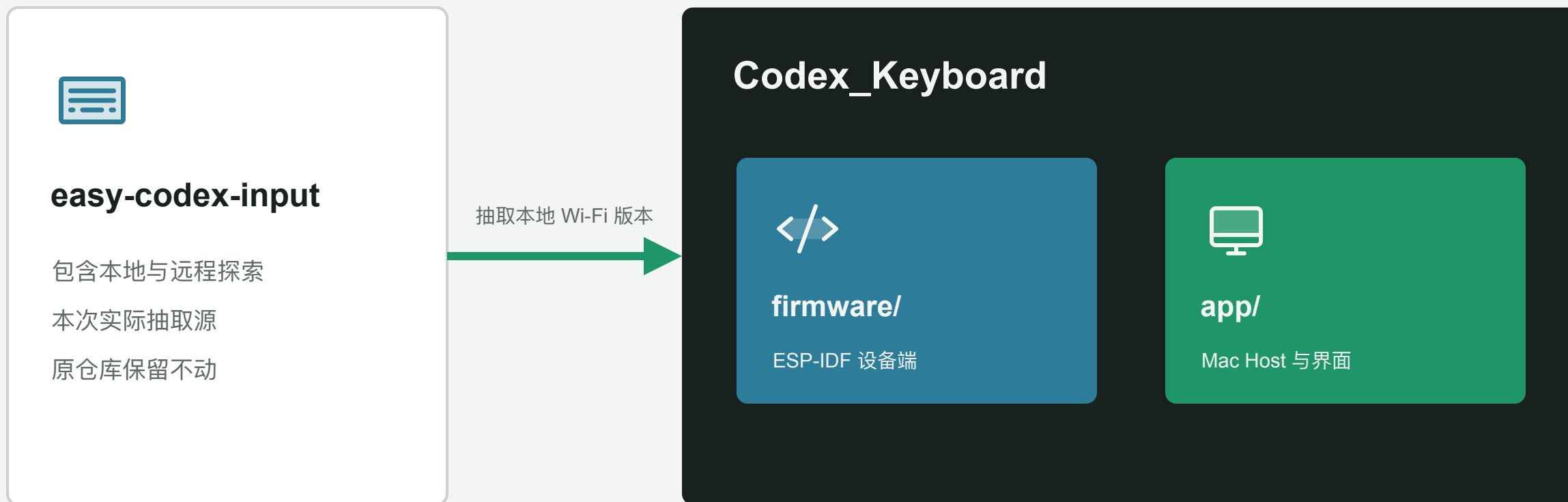


键盘扬声器

边收边播，减少等待

音频在 Mac 上缓存；键盘按下后，通过局域网流式取回。

从现有键盘，抽出自己的 **App** 和固件



更早的固件基线来自 easy-input-keyboard；来源 commit 与许可见仓库记录。

第一次卡住：能烧录，不等于能播放



进不了下载模式

症状：串口只在上电瞬间出现。

原因：这块板的 BOOT 时序不稳定。

做法

先拉住 debug reset，再抓断电上电瞬间。



有开机音，却没有任务音频

这只能证明扬声器和功放能响。

它不证明 UDP、缓存、解码都通。

做法

沿“请求 → 传输 → 解码 → 扬声器”逐段查。

更难的是那些“看起来差不多”的问题



每次都要等很久

UDP 窗口大过设备队列，
丢包后反复重传。

修：窗口与 **IWIP** 队列对齐



灯从右往左亮

代码里的第 1 颗，
不是 PCB 正面的最左灯。

修：按真实灯带链路反向映射



任务完成，S7 却不亮

总结内容其实可用，
却被过度门禁反复拒绝。

修：质量交给 **Luna**，只留运行底线

测试通过，不等于耳朵真的听到了



记录具体槽位、操作和结果；一次可听，不代表并发、断线和重启都已通过。

学员还能把它改成什么

会议任务遥控器

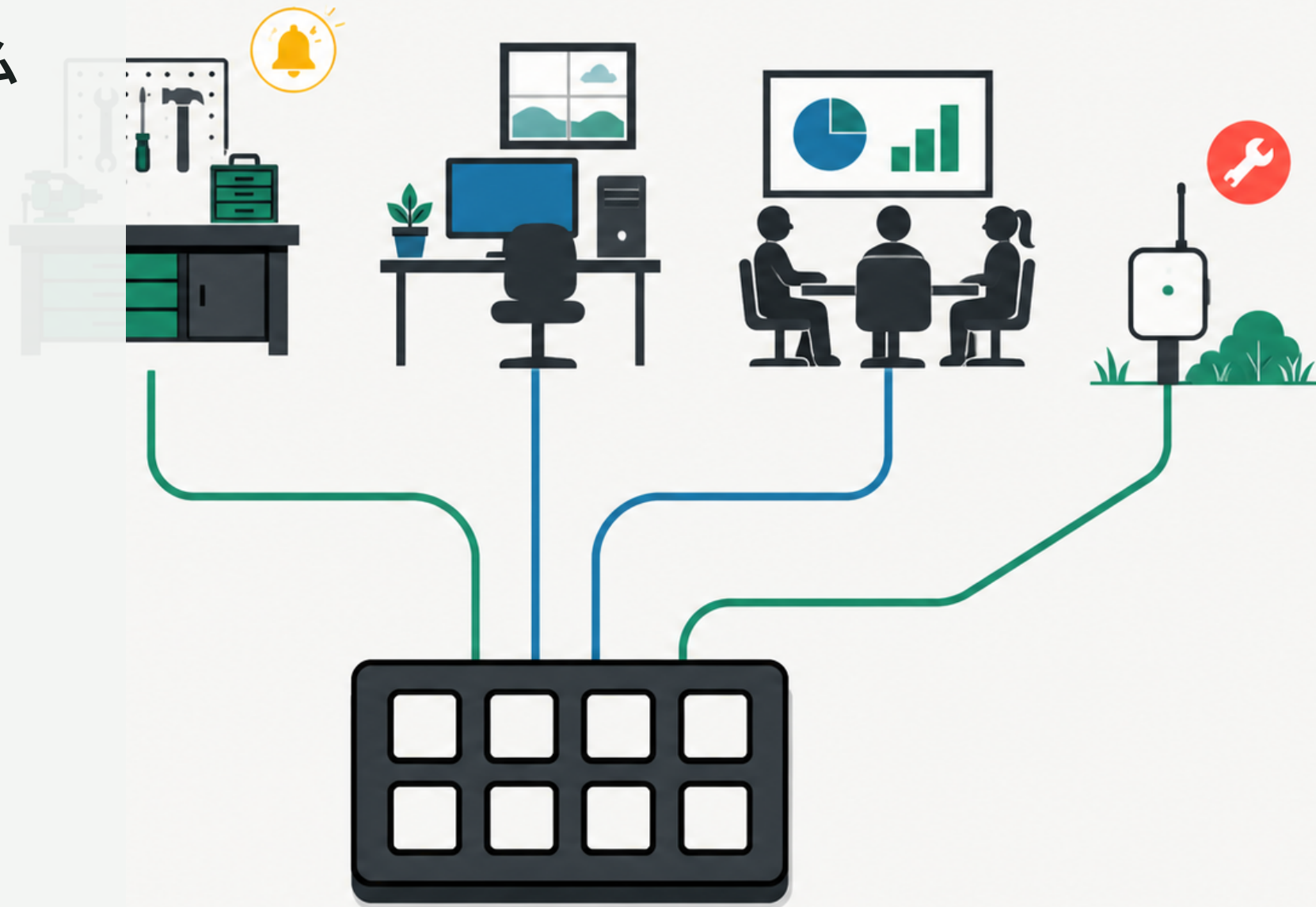
工位语音信箱

设备调试台

家庭自动化控制台

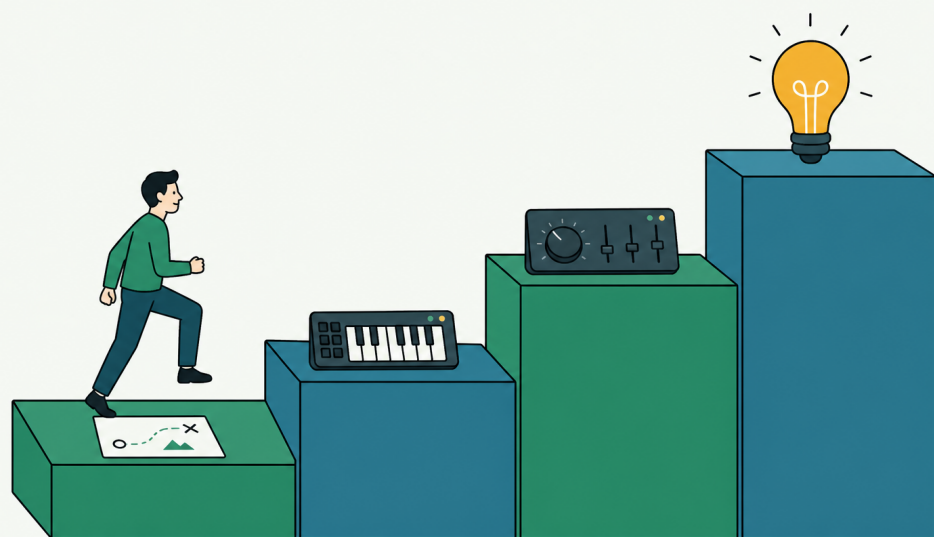
先找一个每天都在重复的动作，
再决定它该长什么样。

概念示意（AI 生成），这些改造不属于当前已实现功能



复现路线图：从看懂到改造

按设备和时间选择练习；提交什么、验证到哪一步，都写清楚。



概念示意（AI 生成）

L0

看懂链路（没有板子也能做）

画出语音输入和结果播放两条链

L1

复现跑通

交验收记录，另附一段真机演示

L2

参数级改造

改音色或总结侧重点，做前后对比

L3

语义级改造

换执行器，交设计说明和真机演示

先核对 Mac、账户、板型和网络，再按飞书教程分步复现。



从一个任务开始， 把说和听都跑通。

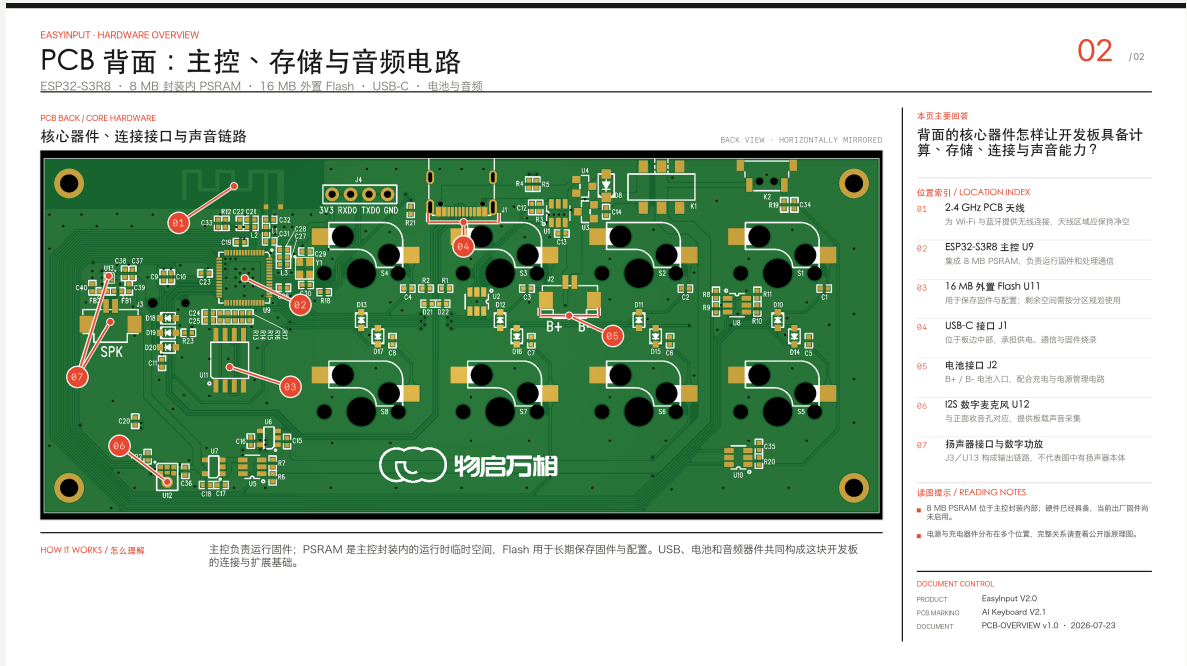
下一步，再选一件你常做的事来改。

留下代码版本、操作步骤和实际结果。

遇到问题，就从出错的那一段查起。

Codex 任务电台 · Lark

附录：EasyInput V2 公开硬件参数



EasyInput V2 PCB 背面

ESP32-S3R8

8 MB PSRAM · 16 MB Flash

输入：8 键 + 旋钮旋转/按压

灯光：5 颗 WS2812

音频：I2S 麦克风 + 数字功放

连接：USB-C、BLE、2.4 GHz Wi-Fi

调试：BOOT 入口、UART 触点

板载十档音量、短按播报与扬声器均由自定义固件启用

附录：S1-S8 与 GPIO 对照

按 PCB 正面丝印，从左到右读取。

上排 · 语音输入

S1 · GPIO2

S2 · GPIO47

S3 · GPIO38

S4 · GPIO41

下排 · 收听汇报

S5 · GPIO1

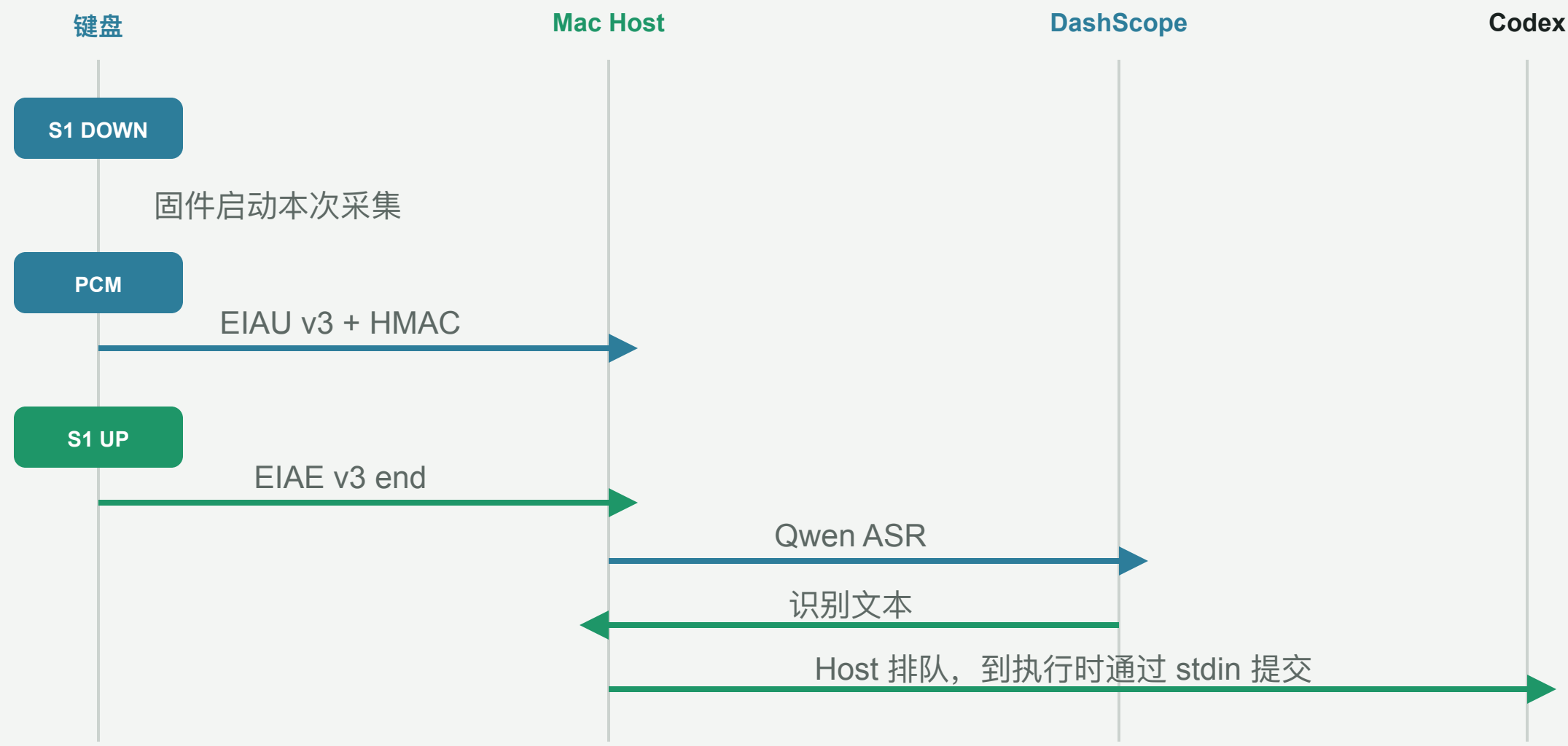
S6 · GPIO6

S7 · GPIO7

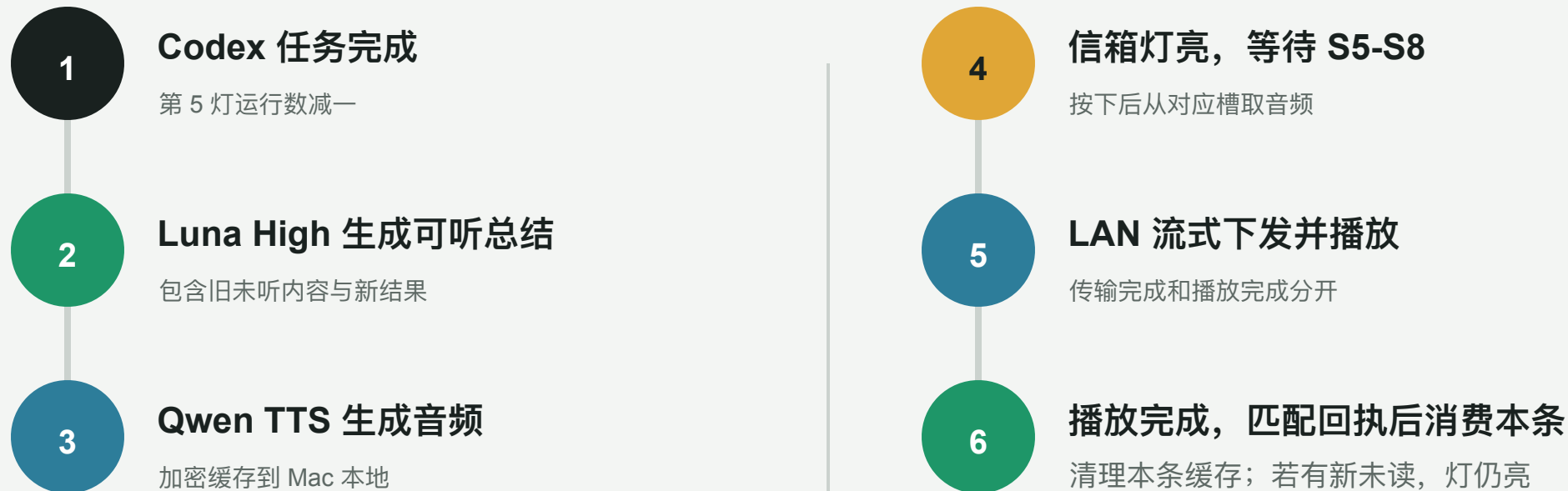
S8 · GPIO48

排查键位时先看丝印和运行诊断，不要根据最终功能反推 GPIO。

附录：完整语音输入时序



附录：总结、缓存与播放时序



所有任务完成后第 5 灯回绿；任何一步失败，仍保留未听内容。

附录：密钥、缓存和数据放在哪里



API Key

只在 Mac 本地 `.env`

文件权限 0600

绝不提交 Git



摘要与音频

Host 管理 SQLite

音频加密缓存到 Mac

听完后精确删除



键盘设备

保存网络、Host 配置和音量

运行时只认槽位号

不保存 Codex 对话

公开仓库只包含代码、文档和脱敏截图，不包含任何真实任务 ID 或密钥。

附录：仓库与测试入口



Larkspur-Wang / Codex_Keyboard

github.com/Larkspur-Wang/Codex_Keyboard

firmware/

ESP-IDF 主工程
host_test / CTest
真机烧录与 HIL

app/

Rust Host
Dashboard UI
SQLite / LAN / TTS

docs/

产品规格
架构与协议
开发过程记录

建议从 README 的同 Wi-Fi 演示路径开始，再看协议与测试。